BSCS 3112 - ARTIFICIAL INTELLIGENCE – LAB

LESSON 6 – CONVERSATIONAL STATE MACHINES AND WORKLOWS

OCTOBER 10, 2026

PRESENTED BY: PROF. ROB MALITAO

LESSON 6 – CONVERSATIONAL STATE MACHINES

Learning Objectives:

Knowledge (K) – Cognitive:

- **Analyze** non-linear customer dialogue flows to identify state transitions (e.g., Greeting →Menu Browsing →Item Customization →Order Confirmation →Payment/Checkout).
- **Explain** the mechanics of Function Calling / Tool Calling in local LLMs to bind natural language requests to dynamic database queries.

LESSON 6 – CONVERSATIONAL STATE MACHINES

Learning Objectives:

Skills (S) – Psychomotor/Practical:

- **Develop** a stateful Python agent using a Finite State Machine (FSM) pattern that tracks customer session memory across multi-turn conversations.
- **Implement** dynamic tool integration where Qwen automatically triggers SQLite database functions (e.g., check_menu_stock(), apply_dietary_filter(), calculate_bill()).
- **Construct** transactional order validation logic that handles user modifications, cancellations, and real-time menu substitutions.

LESSON 6 – CONVERSATIONAL STATE MACHINES

Learning Objectives:

Attitude (A) – Affective:

- **Demonstrate** effective peer collaboration through driver-navigator tandem programming.
- **Value** user experience (UX) and conversational guardrails, ensuring the AI agent remains courteous, coherent, and resilient against customer edge cases.

LESSON 6 – CONVERSATIONAL STATE MACHINES

Subtopic 3.1: Stateful Conversation Architecture

6.1.1: Stateless vs. Stateful Execution: Transitioning from stateless single-turn API calls to persistent multi-turn session state tracking in Python.

- Understanding why raw LLM calls forget previous turns.
- Storing conversation history arrays in Python memory (messages = [...]).

6.1.2: Finite State Machine (FSM) Basics for Dialogue: Structuring Qwen system prompts and JSON schemas to allow the model to autonomously trigger external Python functions and SQLite queries.

- Defining discrete conversation states (GREETING, ORDERING, REVIEW, CONFIRMED).
- Creating a Python state controller variable to track the current phase of the interaction.

LESSON 6 – CONVERSATIONAL STATE MACHINES

Subtopic 6.2: Local LLM Function Calling & Tool Binding

6.2.1: Tool Schema Definition

- Writing JSON specifications for Python functions so Qwen understands *when* and *how* to call them.
- Specifying required function parameters (e.g., item_name: str, quantity: int, special_instructions: str).

6.2.2: Prompt-Based Tool Routing in Qwen

- Injecting available tool signatures into Qwen's system prompt.
- Parsing Qwen's structured JSON response to detect when a tool call is requested vs. when standard conversational text is returned.

LESSON 6 – CONVERSATIONAL STATE MACHINES

Subtopic 6.2: Local LLM Function Calling & Tool Binding

6.2.3: Execution & Feedback Loop

- Executing the corresponding Python function based on the LLM's requested tool call.
- Feeding the function's return value (e.g., {"status": "success", "new_total": 450.00}) back to Qwen so it can formulate a natural conversational response to the user.

LESSON 6 – CONVERSATIONAL STATE MACHINES

Subtopic 6.3: Menu Knowledge Representation & Constraint Handling

6.3.1: Relational Database Modeling for Dining

- Designing SQLite tables for menu_items, categories, modifiers (e.g., extra sauce, size options), and inventory_stock.
- Building a active cart_items table linked to a specific table_session_id.

6.3.2: Inventory & Stock Verification

- Writing SQL query tools (check_availability()) to verify if an item has enough stock before allowing the LLM to add it to the order.
- Handling out-of-stock scenarios gracefully by prompting Qwen to suggest alternative items from the same category.

LESSON 6 – CONVERSATIONAL STATE MACHINES

Subtopic 6.3: Menu Knowledge Representation & Constraint Handling

6.3.3: Item Modification & Customization Parsing

- Capturing special instructions (e.g., *"No mayo"*, *"Extra spicy"*) and attaching them to specific order line items in SQLite.

LESSON 6 – CONVERSATIONAL STATE MACHINES

Subtopic 6.4: Dialogue Guardrails & Edge-Case Handling: Managing conversational interruptions, order cancellations, out-of-stock items, and out-of-domain customer queries.

6.4.1: Order Modification & Deletion Logic

- Handling user requests to remove items (*"Cancel the fries"*) or adjust quantities (*"Make that two burgers instead of one"*).
- Executing SQL UPDATE and DELETE operations safely via tool calls.

6.4.2: Out-of-Domain & Interrupt Handling

- Enforcing system prompts that keep the virtual waiter focused on restaurant-related queries.
- Redirecting non-restaurant questions (e.g., *"What is the capital of France?"*) back to menu assistance politely.

LESSON 6 – CONVERSATIONAL STATE MACHINES

Subtopic 6.4: Dialogue Guardrails & Edge-Case Handling: Managing conversational interruptions, order cancellations, out-of-stock items, and out-of-domain customer queries.

6.4.3: Order Summary & Transaction Finalization

- Generating an itemized receipt with subtotal, tax/service calculations, and final table confirmation.
- Committing the final order state to SQLite and resetting session memory for the next customer.

LESSON 6 – CONVERSATIONAL STATE MACHINES

Subtopic 6.5: Voice I/O Decoupling & Architecture Integration (Bridge to Final Project)

6.5.1: Input/Output Abstraction Layers

- Wrapping the core agent in generic get_input() and send_output() functions.
- Ensuring the core state machine and tool calling engine run identically whether input comes from a text terminal or a speech-to-text (STT) stream.

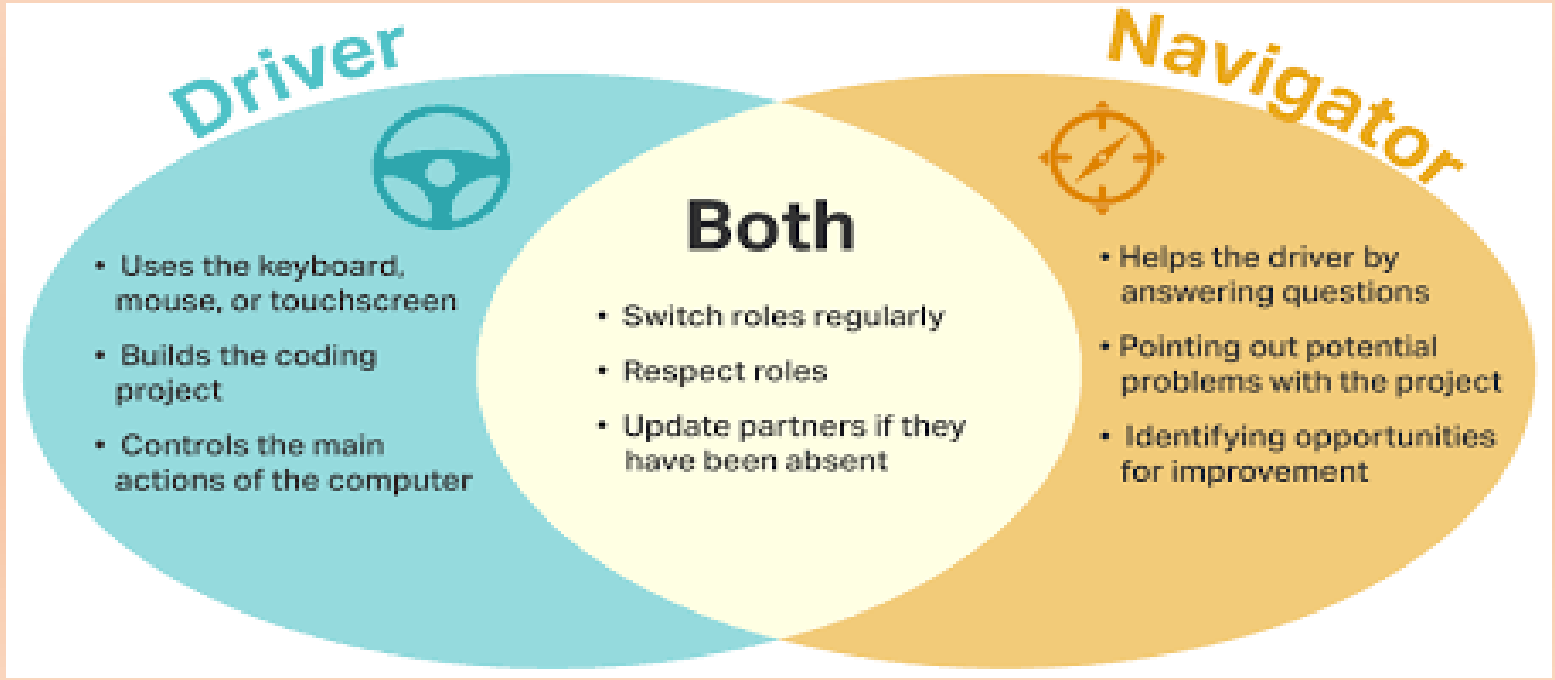
6.5.2: Speech Pipeline Integration Readiness

- Re-introducing their Speech-to-Text (STT) and Text-to-Speech (TTS) modules from the Prelim project as drop-in wrappers around the Module 3 state machine.

LESSON 6 – CONVERSATIONAL STATE MACHINES

Naming partner as driver and navigator in Lab Activity:

- The terms Driver and Navigator originate from the Pair Programming paradigm (popularized by Extreme Programming and Agile software development). The naming logic relies on a direct analogy to rally racing or road trips, where two people share responsibility for completing a journey safely and efficiently.



LESSON 6 – CONVERSATIONAL STATE MACHINES

Responsibilities of Driver vs. Navigator in Pair Programming. Source: VEX Library - VEX Robotics

1. The Core Analogy: Driving a Vehicle

In a road trip or rally race:

- The Driver controls the physical steering wheel, pedals, and gear shift. They focus on tactical execution—staying in the lane, avoiding immediate hazards, and managing current mechanics.
- The Navigator holds the map or GPS, reads terrain notes, and watches for upcoming turns. They focus on strategic direction—ensuring the team gets to the destination without taking wrong turns or running out of fuel.

How the Logic Translates to Software Engineering & AI Projects

Attribute	The Driver	The Navigator
Physical Role	Has hands on the keyboard and mouse.	Watches the monitor, talks, and takes notes.
Mental Focus	Tactical (Micro-Level): Writes clean syntax, handles immediate logic, executes terminal commands, and fixes local typos.	Strategic (Macro-Level): Reviews architecture, checks compliance with requirements/rubrics, catches edge cases, and plans next steps.
Primary Question	"How do I implement this specific line of code or SQL query right now?"	"Does this function solve our overall business goal, and what edge cases might break it?"
Cognitive Load	High focus on syntax, API signatures, and mechanical implementation details.	High focus on design logic, guardrails, schema consistency, and project timeline.

LESSON 6 – CONVERSATIONAL STATE MACHINES

Why This Separation of Concerns Is Pedagogically Powerful

- Eliminates the "Pass-the-Keyboard" Chaos: Without defined roles, two developers sharing one computer often end up either arguing over the keyboard or having one passive person zoning out.
- Dual-Scope Problem Solving: It allows two programmers to solve a single problem at two different levels of abstraction simultaneously (micro-implementation + macro-architecture).
- Continuous Code Review: The Navigator acts as a real-time code reviewer, catching typos, logic errors, and missing guardrails before the code is ever committed or executed.
- Active Role Rotation: Roles are swapped frequently (e.g., every 30–60 minutes or at major project milestones) so both partners develop both hands-on coding skills and high-level architectural thinking.

LESSON 6 – CONVERSATIONAL STATE MACHINES

Lab Activity:

Please refer to the separate sheets for details.